Here's the production architecture I'd use for **Monetchat on Runpod**.

Recommended target state

Use **Runpod Serverless endpoints** for inference, not a long-running Pod as your primary production path. Runpod's Serverless model gives you endpoints, autoscaled workers, and job monitoring; Pods are better for development, admin tasks, and one-off maintenance.

Core layout

```
Web / Mobile App
      |
      v
Monetchat App API (Next.js/Node)
  - auth
  - business logic
  - rate limits
  - orchestration
  - logging
      |
      +--------------------+
      |                    |
      v                    v
Runpod Serverless      Data Services
Inference Endpoints    - Postgres
- chat                 - Redis
- vision               - Qdrant
- embeddings           - Object storage / CDN
```

Split inference into 3 endpoints

Do **not** put all models behind one worker. Create separate Runpod endpoints for:

- **Chat endpoint** → Qwen
- **Vision endpoint** → LLaVA
- **Embedding endpoint** → Nomic embed

Each Runpod endpoint is independently configurable for compute and scaling, which is exactly why this split helps with cost and reliability. Endpoints are the access point, and workers are separate containers managed by Runpod.

Why this split works:

- chat traffic is frequent and latency-sensitive
- vision jobs are heavier and bursty
- embedding requests are short but high volume

Use the right Runpod endpoint type

For Monetchat:

- Use **load balancing Serverless endpoints** for **chat** and probably **embeddings**, because they route directly to available workers and avoid the queue-oriented flow, which is better for low-latency APIs.
- Use either **load balancing** or **queue-based** for **vision**, depending on whether you want instant responses or are okay with queued analysis for heavier image jobs. Queue-based endpoints are the standard Serverless pattern with handler functions.

Storage strategy

Runpod gives you:

- Pod volume disk for Pod persistence
- network volumes for persistent storage independent of Pods or Serverless workers
- S3-compatible access to network volumes for file management without launching a Pod.

For production, I'd use storage like this:

On Runpod

- **Network volume** only when you truly need shared persistent model/data storage across workers, or for very large assets. It persists even when workers scale to zero, but adds network latency and restricts you to the volume's data center.
- Prefer **cached or baked-in models** for the hottest paths when possible, because Runpod notes these load faster than pulling from a network volume.

Outside Runpod

- Store user-uploaded images in object storage/CDN
- Store structured app data in Postgres
- Store vectors in Qdrant
- Store sessions/cache/rate-limit counters in Redis

Environment split

1) Dev / maintenance layer

Keep **one Runpod Pod** for:

- SSH debugging
- validating model updates
- backfilling embeddings
- emergency manual testing

Pods support persistent /workspace volume storage, and optional network volumes if needed.

2) Production inference layer

Use **Runpod Serverless endpoints** for all live user inference. Serverless workers start and stop automatically based on demand.

Monetchat request flow

Chat-only

1. User sends text
2. Monetchat API authenticates and rate-limits
3. API calls **chat endpoint**
4. Response returned to user
5. Conversation metadata saved in Postgres / Redis

Image-assisted search

1. User uploads image
2. Image stored in object storage
3. API calls **vision endpoint** for description/tags
4. API calls **embedding endpoint** on the image-derived description
5. Query Qdrant for nearest products
6. API optionally calls **chat endpoint** to produce the final conversational answer

Seller listing flow

1. Seller uploads item image
2. Vision endpoint generates structured attributes
3. Chat endpoint normalizes title/description
4. Embedding endpoint creates vectors
5. Save listing in Postgres and vectors in Qdrant

Suggested service boundaries

Monetchat App API

Own this in your app:

- users, auth, roles
- product/listing workflows
- payments, moderation, audit logs
- orchestration across Runpod endpoints
- retries, fallback logic
- feature flags and analytics

Runpod workers

Keep them narrow:

- load model
- validate input
- run inference
- return minimal structured JSON

That separation makes the AI layer replaceable later.

Model-specific deployment advice

Chat endpoint

- Dedicated endpoint with autoscaling
- Keep at least one warm worker if latency matters
- Stream tokens from your app only if your chosen worker pattern supports it cleanly

Vision endpoint

- Dedicated heavier GPU profile
- Consider queue-based jobs if image analysis becomes expensive
- Return structured JSON, not prose only

Embedding endpoint

- Smallest practical GPU/worker config
- Batch where possible from the Monetchat API
- This endpoint may become your highest request volume

Runpod explicitly supports endpoint-specific compute and scaling configuration, so this model-per-endpoint design maps well to the platform.

Networking and security

- Do not expose raw Pod ports publicly for production inference
- Put all user traffic through your Monetchat API first
- Store Runpod API credentials server-side only
- Use signed upload URLs for images
- Enforce per-user and per-IP throttling in your API
- Keep internal maintenance Pods on private/admin-only access

If you keep multiple Pods for admin or data tooling, Runpod's global networking can privately connect Pods within your account.

Cost-optimized production path

Best balance for your current stage:

- 1 small admin/debug Pod
- 3 Serverless endpoints: chat, vision, embeddings
- no network volume at first unless model/data size forces it
- use cached/baked models for hot paths
- object storage for uploads
- Qdrant outside the inference workers

Why:

- Serverless scales workers up and down automatically.
- Network volumes are persistent, but they constrain endpoint location and add latency, so they should be used selectively.

Phased rollout

Phase 1

What you can do now:

- keep current Pod for testing
- build the 3 inference services in your app
- standardize JSON contracts

Phase 2

- containerize each worker separately
- deploy 3 Runpod Serverless endpoints
- move production traffic to endpoints

Phase 3

- add autoscaling tuning
- benchmark cold starts
- decide whether any endpoint needs cached models vs network volume
- add fallback behavior if one model is degraded

Runpod's Serverless development docs recommend local testing, then deploying and optimizing endpoints as a normal workflow.

Final recommendation

For **Monetchat**, I would ship this architecture:

- **Next.js/Node app API** as the single public backend
- **Runpod Serverless chat endpoint** for Qwen
- **Runpod Serverless vision endpoint** for LLaVA
- **Runpod Serverless embedding endpoint** for Nomic
- **1 Runpod Pod** kept only for admin/debug/model maintenance
- **Postgres + Redis + Qdrant + object storage** outside the workers
- **No network volume initially**, unless model packaging or dataset size forces it

That gives you better scaling, lower ops burden, and cleaner separation than using one always-on Ollama Pod for everything. It also aligns closely with how Runpod positions Pods, endpoints, workers, storage, and load-balanced inference.

I can turn this into a concrete deployment blueprint next: endpoint list, worker responsibilities, env vars, and request/response contracts.